PROJECT PROPOSAL

Medixa

A Consent-Scoped, Auditable Medical-Data Exchange Layer

with a Smart Hospital OPD System as the Reference Implementation

Submitted To:
[Supervisor's Name]
[Designation]
Department of Computer Science and Engineering
Khulna University of Engineering and Technology (KUET)

Submitted By (Team of 2):
Plabon Barua — [Student ID]
[Partner's Name] — [Student ID]
Department of Computer Science and Engineering, KUET

Date of Submission: [Insert Date]

1. Executive Summary

Most hospital software is built the same way: pick one hospital, put its patients, doctors, and records in one database, and wrap a dashboard around it. Medixa deliberately does not start there. The real, general problem in Bangladesh's healthcare system is not "a hospital needs software" — it is that patient data lives fragmented across many independent, mutually-distrusting parties (hospitals, diagnostic labs, individual doctors, and now AI assistants), with no shared mechanism for a patient to control who sees what, for how long, and to prove afterward what was accessed.

Medixa is therefore designed first around a consent-scoped, auditable data-exchange layer — an internal architecture for sharing sensitive records across roles using time-boxed access grants, patient-held consent tokens, and an immutable access log — and second as a smart hospital OPD system built on top of that layer, serving as its reference implementation. This ordering matters: solving data access as the core problem yields a system whose consent-and-audit core is portable to a second facility as an integration exercise, rather than a single-hospital CRUD app that would need re-architecting later to interoperate with anyone else. The reference implementation is built using Laravel, MySQL, WebSockets (Pusher Channels), and AI API integrations, following the Model-View-Controller (MVC) architectural pattern, with the consent and audit layer sitting beneath every data access in the system.

The project is undertaken by a two-member team, with work divided along vertical, module-based tracks (Section 9) so both members can build in parallel without one waiting on the other, while sharing a single, jointly-owned data and consent model from Week 1 onward.

2. Problem Statement

2.1 The General Problem Beneath the Symptoms

Instead of framing this as "build a hospital management system," the underlying engineering problem is: how do multiple independent, mutually-distrusting parties — patients, doctors, diagnostic labs, hospital administrators, and an AI assistant — share and act on sensitive medical data, without any single party having to fully trust the others with unrestricted access to it? Every symptom below is a visible consequence of no one having solved that underlying problem in Bangladesh's healthcare context yet.

2.2 Context: Bangladesh Healthcare Challenges

In the current healthcare scenario of Bangladesh, both patients and hospital authorities face critical challenges that remain largely unaddressed by existing digital solutions:

- Repetitive Physical Visits for Lab Reports: Patients who undergo morning diagnostic tests must return in the evening — often through severe city traffic or traveling long distances from rural areas — merely to collect paper reports and show them to the doctor, costing 2–3 hours of unnecessary daily travel per visit.
- Double Consultation Expenses & Financial Burden: Patients are often forced to pay an additional consultation fee or book another physical appointment just to have a doctor read a routine lab report — a real burden for lower-income families.
- Severe Queue Congestion & Waiting Time: Lack of transparency regarding doctor arrival times or queue status forces patients to sit in overcrowded waiting rooms for 3–4 idle hours simply to avoid missing their serial.
- Fragmented & Damaged Health Records — the root cause: Patients carry paper prescriptions and physical test reports across clinics, resulting in lost records and incomplete histories. This is not simply "no EHR" — no two providers share a common, patient-controlled identity for the same patient, so every clinic's records exist as an isolated island, leading to repeated tests and compromised treatment quality.
- Post-Consultation Communication Barrier: Once a patient leaves the hospital, asking simple follow-up questions requires a full physical re-visit.
- No Auditability of Sensitive Data Access: Even where digital systems exist, patients typically have no way to see who accessed their medical data, when, and why — access control, where it exists at all, is enforced only at write-time, with no patient-visible trail afterward.

3. Proposed Solution

3.1 Core Design Principle: Consent-Scoped Data Exchange

Rather than granting doctors, admins, or the AI assistant standing access to a patient's full record (the default pattern in most hospital software), Medixa treats every access to sensitive data as a capability — a specific, time-boxed, revocable grant issued by the patient (or automatically by policy for emergencies) that names exactly which resource, which party, and which expiry applies. This single design decision is what turns Medixa from "a hospital's database with a login page" into a data-exchange core that a second facility could plug into as an integration exercise rather than a rewrite — a property we scope carefully in Section 7 to what a two-member, one-semester build can actually demonstrate.

- Consent Tokens, not standing permissions: A lab-report grant might read: "Dr. Rahman may view Patient #4021's CBC report, issued at consultation-booking time, expires in 30 minutes." No blanket "doctors can read all linked patients" rule exists in the core model.
- Patient-Controlled Record Graph: Instead of one hospital owning the canonical patient record, each provider writes an append-only, signed entry into the patient's own record graph. The patient's identity — not any single hospital's database — is the anchor that makes records portable across facilities from day one.
- Immutable, Patient-Visible Audit Log: Every read of a sensitive record is itself logged as an event the patient can view: who accessed it, through which consent token, and when. This makes privacy a checkable property of the system, not a promise in a privacy policy.
- Privacy-Preserving AI Assistant: The AI chatbot never receives patient-identifying data during the conversation itself; it operates only on ephemeral, scoped tokens (e.g., "an available slot with Dr. X on Thursday") and hospital-level metadata.

Where identity re-enters the booking flow

Because a real appointment must ultimately map to a real patient, identity is deliberately re-introduced at one narrow, well-defined point rather than never: the chatbot negotiates and locks a slot using only an ephemeral hold-token; the patient's authenticated session (already logged in, or a lightweight OTP-verified phone number for guests) is attached to that hold-token only at the final confirmation step, server-side, outside the AI provider's request path. The AI model itself is never shown the patient's name, phone number, or record — only "hold-token #A19F confirmed." This closes the gap between the identity-blind claim and the practical need for someone to show up for the appointment.

Consent Expiry and Live Sessions

Expiry is enforced per-request, not per-session: a doctor's browser tab may stay open past a token's expiry, but `ConsentService` re-validates the token's `expires_at` on every single request that touches the resource, so an already-open tab loses access the moment the underlying token expires — no cached session is trusted to outlive its token.

3.2 System Overview — The Reference Implementation

On top of this consent-and-audit foundation, Medixa's reference implementation is a smart hospital OPD system delivered through a multi-tiered, role-based architecture comprising four interconnected portals:

Portal	Primary Users	Key Functionalities
Public Portal	General Public	Hospital directory, doctor profiles, AI chatbot, instant appointment booking
Patient Portal	Registered Patients	EHR access, live queue tracking, lab report inbox, doctor assistance requests

Portal	Primary Users	Key Functionalities
Doctor Portal	Healthcare Providers	Patient queue management, pending report reviews, e-prescribing, patient history
Admin Panel	Hospital Administrators	Doctor/department management, schedule oversight, system usage monitoring

3.3 Key Features

3.3.1 Public Portal & Privacy-Preserving AI Assistant

- Hospital Directory: department, doctor profile, room number, emergency-service, and facility information.
- AI Assistance & Auto-Booking: an integrated chatbot (OpenAI/Claude API) that answers hospital-related queries and completes booking workflows from parsed user intent.
- Identity-Blind by Design: the chatbot's API layer strips patient-identifying fields before any request reaches the AI provider — it negotiates against ephemeral hold-tokens, with identity attached only at final confirmation (Section 3.1).

3.3.2 Live OPD Queue Tracker (Zero-Waiting Strategy)

- Real-Time Serial Monitoring: "Doctor Present | Current Serial: #14 | Your Serial: #22" on the patient dashboard.
- Smart Travel Planning: patients track doctor movement and time their arrival, targeting a meaningful reduction in waiting-room crowding (validated post-deployment; see Section 8.2 for how this is measured).
- Technology: real-time updates via WebSockets (Pusher Channels).

3.3.3 Automated Lab Report Delivery & Remote Doctor Review

- Direct Profile Upload: diagnostic departments upload results directly to the patient's record graph as a signed entry.
- Consent-Gated Doctor Inbox: reports route to the assigned doctor's "Pending Report Reviews" list alongside an auto-issued, time-boxed consent token scoped to that report.
- Remote E-Prescription & Follow-Up: doctors review during allocated slots and issue updated e-prescriptions directly to the patient's profile.
- Selective In-Person Calling: physical visits are requested only when a report indicates a condition requiring immediate clinical intervention.

3.3.4 Patient Portal, Record Graph & Consent Center

- Patient-Controlled Record Graph: a complete, patient-anchored timeline of diagnoses, prescriptions, lab reports, and doctor notes, portable across hospitals.
- Consent Center: a dedicated screen to view, grant, and revoke active consent tokens, and browse the access audit log.
- Doctor-Patient Linkage: linked-profile view with non-emergency messaging/assistance requests.

3.3.5 Doctor Workstation & Analytics Dashboard

- Patient Workload Metrics: live totals for patients served, pending appointments, and pending report reviews.

- Consent-Scoped History Inspection: a connected patient's visible history is limited to what an active consent token covers.
- Instant Action Alerts: real-time notifications for incoming assistance requests or emergency updates.

3.3.6 Admin Panel & Ecosystem Supervision

- Operational management for doctors, departments, schedules, and facility activity.
- Audit Oversight: admins review aggregate (not individual-record) access patterns to flag anomalous or excessive data requests.

4. System Architecture & Technology Stack

4.1 Architecture Overview

The platform follows the Model-View-Controller (MVC) architectural pattern for separation of concerns, maintainability, and scalability. Beneath the MVC application layer sits the consent-and-audit layer — every controller that touches patient data routes through a single `ConsentService`, so access control is enforced structurally rather than re-implemented per feature. Concretely: a request for a resource without a valid, unexpired consent token is rejected by middleware before it reaches controller logic at all.

Chosen stack (finalized, no longer dual-tracked): Laravel (PHP) on the backend and MySQL as the relational store. This matches the project structure below, the team's existing tooling (XAMPP/Laragon), and avoids splitting a two-member team's limited time across two backend runtimes.

Project Structure

```
medixa-hospital-ecosystem/
├── app/
|   ├── Controllers/  (AuthController, PatientController, DoctorController, LabReportController, QueueController, AppointmentController, ChatbotController, ConsentController)
|   ├── Models/  (User, PatientProfile, DoctorProfile, Appointment, Prescription, LabReport, QueueStatus, AssistanceRequest, ConsentToken, AuditLogEntry)
|   └── Services/  (AIService, NotificationService, QueueService, ConsentService, AuditService)
├── database/  (migrations, seeders)
├── resources/views/  (public/, patient/, doctor/, admin/)
└── routes/  (web.php, api.php)
```

4.2 Technology Stack

Component	Technology	Purpose
Backend Framework	Laravel (PHP)	MVC architecture, routing, business logic
Database	MySQL	Relational data storage with foreign-key constraints
Frontend	HTML5, CSS3, JavaScript, Tailwind CSS	Responsive, modern UI/UX
Real-time Updates	Pusher Channels (WebSockets)	Live queue tracking, instant notifications
AI Integration	OpenAI / Claude API	Conversational chatbot, intent parsing
Authentication	Laravel Sanctum	Secure, session/token-based, role-aware access control
File Storage	Private cloud storage (S3/DigitalOcean Spaces)	Secure lab-report storage with signed URLs
Hosting	Railway / Render	Cloud deployment for the live demo

4.3 Database Schema (Core Entities)

Table	Key Fields	Purpose
Users	id, name, email, password, role	3-role authentication base
Patient_Profiles	id, user_id, dob, blood_group, emergency_contact, medical_history	Patient-specific data
Doctor_Profiles	id, user_id, department_id, qualification, designation, patients_seen_count	Doctor-specific data
Appointments	id, patient_id, doctor_id, appointment_date, serial_number, status	Appointment management
Live_Queues	id, doctor_id, current_running_serial, doctor_present_status, updated_at	Real-time serial tracking
Lab_Reports	id, patient_id, doctor_id, report_title, file_path, review_status, uploaded_at	Report management with routing
Prescriptions	id, appointment_id, patient_id, doctor_id, diagnosis, medicines, instructions	Digital prescription records
Assistance_Requests	id, patient_id, doctor_id, message, status, created_at	Patient-doctor messaging
Consent_Tokens	id, patient_id, grantee_id, resource_type, resource_id, issued_at, expires_at, revoked_at	Scoped, time-boxed access grants
Audit_Log	id, actor_id, patient_id, resource_type, resource_id, consent_token_id, accessed_at	Immutable, patient-visible access trail

5. Security & Data Privacy Framework

Medixa handles sensitive medical data (EHR, lab reports, prescriptions), so privacy is the system's core architectural constraint (Section 3.1), not an add-on layer. Role-based rules exist as a coarse first filter — the binding decision on every request is whether a valid, unexpired consent token exists.

5.1 Consent-Scoped Access Control

- Token-Gated, Not Role-Gated: a patient can only view their own profile, reports, and prescriptions; a doctor's access additionally requires a live, unexpired consent token for that specific resource — role alone never grants access. URL manipulation (e.g., patient/10 → patient/11) is blocked by middleware regardless of role.
- Automatic, Per-Request Expiry: tokens expire on a short, resource-appropriate window; expiry is checked on every request, so an already-open browser tab loses access the instant its token lapses (Section 3.1).
- Middleware Security: every request validates the user's role and an active consent token before returning data, and every successful access is written to the audit log by `AuditService`.

5.2 Authentication & Session Security

- Password Hashing: Bcrypt/Argon2id — never plain text.
- Laravel Sanctum: secure, session/token-based authentication for API requests, including AI chatbot interactions.
- Two-Factor Authentication (2FA): optional email/SMS verification for doctor and admin logins (v2 — scheduled in Section 10, Week 10).

5.3 Data Encryption

- In-Transit: TLS/SSL (HTTPS) everywhere.
- At-Rest: sensitive fields (NID, mobile numbers, prescription notes) encrypted with AES-256 (v2 — scheduled in Section 10, Week 10).

5.4 Secure File Storage

- Private storage bucket — lab reports and documents are never in public folders.
- Signed, temporary URLs for file access, expiring within 1–2 minutes.

5.5 AI & API Security

- Identity-Blind AI Assistant: patient personal information is never sent to the AI API during negotiation; identity is attached only at final confirmation, server-side (Section 3.1).
- Prompt Injection Protection: user inputs are sanitized before reaching the model or being interpreted as commands.

5.6 Web Vulnerability Defense

Vulnerability	Mitigation
SQL Injection	Eloquent ORM / prepared statements
Cross-Site Scripting (XSS)	HTML escaping and input sanitization
CSRF Attacks	Required CSRF tokens on all forms

Vulnerability	Mitigation
Brute-Force / Credential Stuffing	Rate limiting on login and chatbot endpoints

6. Social & Economic Impact in Bangladesh

6.1 Direct Patient Benefits

- Reduced daily travel and waiting time per visit.
- Fewer redundant consultation fees for routine lab-report reviews.
- Continuity of care through a universal, patient-anchored record — no more lost paper records.
- Reduced physical strain for elderly and chronically ill patients.

6.2 Systemic Benefits

- Hospital decongestion through live queue visibility.
- Reduced unnecessary trips and associated city traffic.
- Direct alignment with Smart Healthcare and Digital Bangladesh objectives.

Note: the percentage figures in earlier drafts (e.g., wait-time or congestion reduction) describe the system's intended real-world impact once deployed at scale with real patients. Within the academic project timeline, these are treated as projected impact, not measured outcomes — Section 8.2 separates them from the technical KPIs the team will actually verify during the project.

7. Project Rationale & Feasibility

7.1 Why This Project?

Criteria	Medixa's Advantage
Daily Relevance	Every family in Bangladesh faces healthcare access challenges.
Feasibility (2-member team)	Software-only system, no hardware dependency; work is split into two independent module tracks (Section 9) so both members build in parallel rather than serializing on one codebase.
Unique Value	Three features rarely offered together: live queue tracking, remote report review, and an identity-blind AI assistant.
Scalability	MVC architecture with a consent layer beneath it, so the system can grow with hospital and feature needs without re-architecting.

7.2 Risk Assessment

Risk	Mitigation Strategy
API Rate Limits	Caching and request throttling on AI/Pusher calls
WebSocket Disconnection	Fallback to short-interval polling
Data Breach	Multi-layer encryption, consent-gated + role-based access control
Deployment Failure	Multiple hosting fallback options (Railway, Render)
Team Coordination / Merge Conflicts	Module ownership by track (Section 9) limits overlapping edits; shared core files (ConsentToken, AuditLog, migrations) are agreed and frozen together in Week 1; all work merges via feature branches and pull requests reviewed by both members.
Single-Member Unavailability (bus factor)	Both members review each other's core services (ConsentService, AuditService) at each integration checkpoint (Weeks 4, 8, 11) so either member can debug or extend the other's module if needed.
Uneven Workload / Scope Drift	Fixed weekly deliverables per track (Section 10) with joint Sunday check-ins to rebalance tasks before drift compounds.

8. Expected Outcomes & Deliverables

8.1 Core Deliverables

- A fully functional, multi-role web application hosted on cloud infrastructure.
- A working real-time Live Queue Tracker and automated lab-report workflow.
- An integrated, identity-blind AI Assistant for hospital guidance and scheduling.
- Complete technical and user documentation.
- A live demo ready for project defense.

8.2 Technical KPIs (Measured by the Team)

Metric	Target
Consent enforcement coverage	100% of patient-data endpoints reject requests without a valid, unexpired consent token (verified by automated tests)
Audit completeness	100% of successful sensitive-data reads produce a corresponding Audit_Log entry
Queue update latency	< 1 second from doctor-side update to patient dashboard, via WebSocket
Unauthorized cross-patient access (self-test)	0 successful attempts in a deliberate penetration self-test (IDOR, token replay, expired-token reuse)
System uptime during demo window	≥ 99% across the final two weeks of testing

8.3 Projected Real-World Impact (Not Directly Measured In-Project)

- Patient wait-time reduction, unnecessary-visit reduction, and user-satisfaction targets described in Section 6 represent the system's intended impact at real deployment scale, and are explicitly separated here from the technical KPIs above that the team will verify.

9. Team & Work Division

The project is built by a two-member team using a vertical, module-based split rather than a frontend/backend split — each member owns full-stack responsibility (routes, controller, model, view) for their assigned modules, which lets both members work in parallel without blocking on each other. Both members jointly design and freeze the shared data model — Users, ConsentToken, AuditLogEntry, and core migrations — in Week 1, since every other module depends on it.

9.1 Track A — Consent, Audit & Clinical Workstation (Member 1: Plabon Barua)

- ConsentService & AuditService: issuing, validating, expiring, and revoking consent tokens; writing immutable audit-log entries.
- ConsentController, ConsentToken and AuditLogEntry models, and their migrations (co-designed with Member 2 in Week 1).
- Doctor Portal: patient queue management, pending report reviews, e-prescribing, consent-scoped patient history.
- Live OPD Queue Tracker: QueueService, QueueController, Live_Queues table, and the Pusher/WebSocket broadcast logic.
- Admin Panel: doctor/department management, schedule oversight, aggregate audit-oversight views.

9.2 Track B — Identity, Patient Experience & AI (Member 2: [Partner's Name])

- AuthController and 3-role authentication (Sanctum), including the OTP/guest-booking identity path used by the chatbot flow.
- Patient Portal: record-graph timeline, Consent Center (grant/revoke/view audit trail), lab-report inbox.
- Public Portal: hospital directory, doctor profiles, instant appointment booking UI.
- ChatbotController and AIService: intent parsing, ephemeral hold-token negotiation, identity-blind booking flow, and final confirmation handoff (Section 3.1).
- NotificationService: real-time alerts for assistance requests, report-ready events, and emergency updates.

9.3 Shared Responsibilities

- Week 1 data-model and migration design (both members, jointly).
- Integration checkpoints at the end of Weeks 4, 8, and 11 (Section 10) — both members test the other's module against their own.
- Security review pass (Week 9): each member deliberately attempts to break into the other's module (IDOR, expired-token reuse, mass assignment) before the joint fix pass.
- Documentation, deployment, and defense preparation (Weeks 10–12, jointly).

10. Complete 12-Week Roadmap (Two-Member, Parallel Tracks)

★ marks a mandatory joint integration checkpoint where both tracks must merge cleanly before the next phase begins.

Wk	Member 1 (Track A — Consent, Audit, Clinical)	Member 2 (Track B — Identity, Patient, AI)	Week's Success Signal
1	Joint (both members): finalize stack; design & migrate core schema (Users, ConsentToken, AuditLogEntry, Appointments); Public Portal HTML/Tailwind scaffold (M2 leads)	AuthController scaffold (register/login) with Sanctum	Shared schema in Git; both members can run the app locally
2	ConsentService v1 (issue/validate/expire) + AuditService v1 (write-only log); Live_Queues migration + QueueService skeleton	Public Portal completed (directory, doctor profiles, department info)	ConsentService passes unit tests; Public Portal demoable
3	ConsentController (grant/revoke API) + Doctor Portal shell (queue list, patient list)	Patient Portal shell: record-graph timeline (read-only) + Consent Center UI (view tokens)	A doctor can view a patient's record only with a valid seeded token
4 ★	INTEGRATION CHECKPOINT (joint) — Consent flow end-to-end: booking issues a token → doctor sees gated data → patient sees it in Consent Center	(same)	A full consent lifecycle (issue → use → expire) demoable by both members
5	Doctor Portal: e-prescribing, linked patient history inspector (consent-scoped)	AI chatbot: intent parsing + ephemeral hold-token negotiation (no booking yet)	Doctor can write a prescription; chatbot can propose a slot without seeing identity
6	Lab_Reports migration + LabReportController: upload, routing to doctor's pending-review inbox with auto-issued token	Chatbot: final booking confirmation handoff (identity binding, Sec. 3.1) + Appointments write path	Patient receives an uploaded report; chatbot completes a real booking
7	QueueService: live serial broadcast via Pusher; Doctor Portal "present/serial" controls	NotificationService: real-time alerts (assistance requests, report-ready)	Patient dashboard shows live serial updates within 1 second
8 ★	INTEGRATION CHECKPOINT (joint) — Admin Panel (doctor/department management, aggregate audit oversight) + full system smoke test across both tracks	(same)	All 4 portals reachable end-to-end with seeded data
9	Joint: adversarial security review — each member attempts to break the other's module (IDOR, token replay, mass assignment, expired-token reuse)	(same)	Zero successful unauthorized-access attempts in the self-test

Wk	Member 1 (Track A — Consent, Audit, Clinical)	Member 2 (Track B — Identity, Patient, AI)	Week's Success Signal
10	Fix pass from Week 9 findings + AES-256 field encryption (v2) + optional 2FA (v2)	UI/UX refinement pass, responsive testing across devices	Security fixes verified; UI passes a mobile + desktop review
11 ★	INTEGRATION CHECKPOINT (joint) — deployment to Railway/Render, mock-data population, full regression pass	(same)	System is live at a public URL with seeded demo data
12	Joint: documentation (technical + user manual), demo script rehearsal, supervisor review, project defense	(same)	Defense-ready: live demo + docs + Q&A prepared

Phase summary

Phase	Weeks	Goal
Phase 1 — Foundation & Core Consent Model	1–4	Shared schema live; consent-token lifecycle working end-to-end across both tracks.
Phase 2 — Portals, Live Tracker & AI Booking	5–8	All four portals functional; live queue and identity-blind AI booking working.
Phase 3 — Hardening, Deployment & Defense Prep	9–12	Security-reviewed, deployed, documented, and rehearsed for defense.

11. Future Scope

- Telemedicine video consultations for remote appointments.
- Automated SMS alerts for medication reminders and queue status, for feature-phone users.
- Payment gateway integration (bKash/Nagad) for appointments and lab-test payments.
- Native Android/iOS mobile application.
- Multi-hospital support — extending the consent-and-audit core to a second facility as the first real test of its protocol-like portability (Section 3.1).

12. Required Resources & Support

12.1 Development Tools

Tool/Service	Purpose	Cost
VS Code	Code editor	Free
XAMPP/Laragon	Local server environment	Free
Postman	API testing	Free
Git/GitHub	Version control (shared repo, PR-based workflow)	Free

12.2 Third-Party Services

Service	Purpose	Cost
OpenAI / Claude API	AI chatbot	Free tier available
Pusher Channels	Real-time WebSockets	Free tier available
Railway / Render	Cloud hosting	Free tier available
MySQL	Data storage	Free (included)

12.3 Supervisor Support Requested

- Guidance on architectural decisions, especially the consent/audit layer's boundaries.
- Review of the database schema to confirm normalization and the consent-token/audit-log design.
- Feedback on the two-member work division and integration checkpoints.
- Assistance with deployment strategy and server configuration.
- Feedback on UI/UX design from a healthcare perspective.

13. Conclusion

Medixa is deliberately scoped as more than a single hospital's booking system. By building the consent-scoped, auditable data layer first, and delivering the smart hospital OPD system as its reference implementation, the project addresses the general problem underneath Bangladesh's fragmented healthcare data — not just its most visible symptom. This is a practical, feasible, high-impact project for a two-member team on a one-semester timeline: the module-based work division (Section 9) and the phased, checkpointed roadmap (Section 10) are designed so both members contribute full-stack ownership of real, demonstrable modules, while the shared consent-and-audit core remains portable to a second facility as future work rather than a promise made beyond what the team can show.

By combining capability-based access control, an immutable patient-visible audit trail, real-time queue technology, and a privacy-preserving AI assistant, the system has the potential to transform the patient experience, reduce healthcare costs, and contribute to the national vision of Digital Bangladesh — while giving patients, for the first time, a system-level answer to "who has seen my medical data, and why."

The project's software-only architecture keeps implementation risk low, its two-member, module-based team structure keeps the 12-week timeline realistic, and its modular MVC design with a consent layer beneath it leaves the system open to future enhancement without requiring a rewrite.

14. References

1. Bangladesh Bureau of Statistics, Health and Demographic Data, 2023.
2. Ministry of Health and Family Welfare, Digital Bangladesh Healthcare Vision, 2021.
3. Laravel Documentation, Security & Authentication, 2024.
4. Pusher Channels, Real-time WebSocket Implementation Guide, 2024.
5. OpenAI API Documentation, Conversational AI Integration, 2024.

_____

Plabon Barua — Student, Department of CSE, KUET Date: _____

_____

[Partner's Name] — Student, Department of CSE, KUET Date: _____

Supervisor's Remarks & Approval:

_____

[Supervisor's Name], [Designation], Department of CSE, KUET Date: _____

Recommendation: ☐ Approved ☐ Revisions Required ☐ Resubmit